

TIVIFY

Manual Completo de Desarrollo y Operaciones

Version 2.4.0

Plataforma IPTV/OTT Self-Hosted

Backend	Go 1.22 + Fiber v2 + PostgreSQL 16 + Redis 7
Frontend	Next.js 14 + React 18 + TypeScript + Tailwind CSS
Android	Kotlin + Jetpack Compose + ExoPlayer + Hilt
Infra	Docker Compose + nginx + Tailscale VPN

Marzo 2026

Indice de Contenidos

1. Arquitectura del Proyecto
2. Requisitos del Sistema
3. Instalacion y Configuracion Inicial
4. Docker y Contenedores
5. Backend (Go/Fiber)
6. Frontend (Next.js)
7. Android (Kotlin/Compose)
8. Base de Datos (PostgreSQL)
9. Redis y Cache
10. Nginx y Proxy Inverso
11. Tailscale VPN
12. Testing y Coverage
13. Gestion de Versiones
14. Operaciones Diarias
15. Backups y Restauracion
16. Streaming y Media
17. Troubleshooting
18. Referencia Rapida

1. Arquitectura del Proyecto

TIVIFY es una plataforma IPTV/OTT self-hosted compuesta por cuatro capas principales que se comunican a través de una API REST y WebSockets.

1.1 Diagrama de Componentes

```
Cliente (Web/Android) | [nginx :80/:443] <-- Rate limiting, Security Headers, SSL | \
[Frontend] [Backend] <-- API REST + WebSocket Next.js:3000 Go:8080 | \ [PostgreSQL]
[Redis] :5432 :6379 | [FFmpeg] <-- Transcodificacion HLS | [/media volume] <-- Videos,
HLS segments, thumbnails
```

1.2 Stack Tecnológico

Capa	Tecnologia	Funcion
Backend	Go 1.22 + Fiber v2	API REST, Auth JWT, WebSocket, FFmpeg
ORM	GORM	Migraciones automaticas, relaciones
Base de Datos	PostgreSQL 16	Datos persistentes, FTS (Full Text Search)
Cache	Redis 7	Sesiones, cache de categorias, pub/sub
Frontend	Next.js 14 + React 18	SPA con App Router, SSR, PWA
UI	Tailwind CSS + shadcn/ui	Componentes accesibles, responsive
Streaming	hls.js	Reproductor HLS adaptativo (web)
Android	Kotlin + Compose	App nativa con Material 3
Player Android	ExoPlayer / Media3	Reproduccion HLS nativa
DI Android	Hilt (Dagger)	Inyeccion de dependencias
Proxy	nginx	Reverse proxy, rate limiting, headers
VPN	Tailscale	Acceso remoto encriptado
Metricas	Prometheus	Metricas HTTP, contadores custom
i18n	i18next	Internacionalizacion (es/en)

1.3 Estructura de Directorios

```
TIVIFY/ backend/ # API Go + Fiber cmd/server/ # Punto de entrada (main.go) internal/
cache/ # Servicio de cache Redis config/ # Configuracion desde env vars database/ #
Conexiones PostgreSQL + Redis dto/ # Data Transfer Objects handler/ # Controladores HTTP
metrics/ # Metricas Prometheus middleware/ # Auth, CORS, Rate Limit, Logging model/ # 16
modelos GORM repository/ # Capa de acceso a datos router/ # Definicion de rutas API
service/ # Logica de negocio util/ # JWT, Logger, Validacion ws/ # WebSocket hub
frontend/ # Next.js 14 src/app/ # App Router (pages) src/components/ # Componentes React
src/context/ # Auth + Toast providers src/lib/ # API client, utils, i18n __tests__/ #
```

```
Jest tests (99%+ coverage) android/ # App Kotlin/Compose
app/src/main/java/com/tivify/app/ data/api/ # Retrofit API + Interceptors di/ # Hilt
modules ui/ # Screens + ViewModels docker/ # Docker Compose + configs nginx/ # Nginx
config postgres/ # Init SQL tailscale/ # VPN container nginx/ # Nginx Dockerfile +
conf.d/
```

2. Requisitos del Sistema

2.1 Hardware Minimo

Componente	Minimo	Recomendado
CPU	2 cores	4+ cores (para transcoding FFmpeg)
RAM	4 GB	8+ GB
Disco	20 GB (SO + app)	100+ GB (con media)
Red	10 Mbps	100+ Mbps (streaming multiple)

2.2 Software Requerido

Software	Version	Proposito
Docker	24.0+	Contenedores de todos los servicios
Docker Compose	v2.20+	Orquestacion de servicios
Git	2.30+	Clonar repositorio
Node.js	18+ (opcional)	Desarrollo frontend local
JDK 17 (opcional)	17+	Desarrollo Android local

2.3 Puertos Requeridos

Puerto	Servicio	Exposicion
80	nginx (HTTP)	Publico
443	nginx (HTTPS)	Publico
8080	Backend Go	Solo interno (Docker network)
3000	Frontend Next.js	Solo interno (Docker network)
5432	PostgreSQL	Solo interno (Docker network)
6379	Redis	Solo interno (Docker network)

IMPORTANTE: Solo los puertos 80 y 443 se exponen al exterior. PostgreSQL y Redis NO deben ser accesibles desde fuera de la red Docker.

3. Instalacion y Configuracion Inicial

3.1 Instalar Docker

```
# Ubuntu/Debian curl -fsSL https://get.docker.com | sh sudo usermod -aG docker $USER
newgrp docker # Verificar docker --version docker compose version
```

3.2 Clonar el Proyecto

```
git clone <URL_REPOSITORIO> /opt/tivify cd /opt/tivify
```

3.3 Configurar Variables de Entorno

Copiar el archivo de ejemplo y editar con valores de produccion:

```
cp .env.example .env nano .env
```

Variables Criticas de Seguridad

Variable	Descripcion	Ejemplo
BASE_URL	URL publica del servidor	http://mi-servidor.com
DB_PASSWORD	Password de PostgreSQL	P@ssw0rd_Segur0_2024!
REDIS_PASSWORD	Password de Redis	R3d1s_S3cur3!
JWT_SECRET	Clave JWT (min 32 chars)	clave-aleatoria-de-32-caracteres-min
ADMIN_USERNAME	Usuario admin inicial	admin
ADMIN_PASSWORD	Password admin inicial	CambiarEnPrimerLogin!
ADMIN_EMAIL	Email del admin	admin@dominio.com

Variables de Streaming

Variable	Descripcion	Default
FFMPEG_PRESET	Preset de codificacion	faster
FFMPEG_HWACCEL	Aceleracion HW (nvidia/intel/none)	none
FFMPEG_AUDIO_BITRATE	Bitrate de audio	192k
HLS_SEGMENT_DURATION	Duracion segmento HLS (seg)	10
HLS_PLAYLIST_SIZE	Segmentos en playlist HLS	5
LIBRARY_PATH	Ruta a biblioteca de medios	/mnt/usb
TMDB_API_KEY	API key de TheMovieDB	(obtener en tmdb.org)

Variables de Tailscale

Variable	Descripcion	Ejemplo
TS_AUTHKEY	Auth key (efimera recomendada)	tskey-auth-xxxxx
TS_HOSTNAME	Nombre en la tailnet	tivify
TS_SERVE_MODE	Modo: https o proxy	https
ENABLE_TAILSCALE	Habilitar VPN	true

IMPORTANTE: Generar JWT_SECRET con: `openssl rand -base64 48`. NUNCA usar el valor por defecto en produccion.

3.4 Archivo .env Completo

El archivo `.env.example` contiene 152 lineas con TODAS las variables disponibles, agrupadas por seccion: General, PostgreSQL, Redis, Backend, Frontend, Admin, Tailscale, Library Scanner, Docker Runtime, Backup, Network, Security, API, Streaming, FFmpeg, APK, Database y Logging.

4. Docker y Contenedores

4.1 Servicios Docker

Servicio	Imagen	CPU	RAM	Funcion
postgres	postgres:16-alpine	1 core	1024 MB	Base de datos
redis	redis:7-alpine	0.5 core	256 MB	Cache y sesiones
backend	Custom (Go)	1 core	512 MB	API REST
frontend	Custom (Next.js)	0.5 core	256 MB	Interfaz web
nginx	Custom (nginx)	1 core	256 MB	Proxy inverso
tailscale	Custom	-	-	VPN (network: nginx)
android-build	Custom (Gradle)	-	-	Build APK (perfil)

4.2 Levantar Todos los Servicios

```
cd /opt/tivify/docker docker compose up -d # Verificar que todos estan healthy docker  
compose ps # Orden de arranque automatico: # postgres -> redis -> backend -> frontend ->  
nginx -> tailscale
```

4.3 Reconstruir un Servicio

```
# Reconstruir solo el backend cd /opt/tivify/docker docker compose up -d --build backend  
# Reconstruir solo el frontend docker compose up -d --build frontend # Reconstruir TODO  
desde cero docker compose up -d --build --force-recreate
```

4.4 Ver Logs

```
# Logs de todos los servicios docker compose logs -f # Logs de un servicio especifico  
docker compose logs -f backend docker compose logs -f frontend docker compose logs -f  
postgres # Ultimas 100 lineas docker compose logs --tail=100 backend
```

4.5 Parar y Reiniciar

```
# Parar todo (conserva datos) docker compose down # Parar y BORRAR volúmenes (PELIGRO:  
pierde datos) docker compose down -v # Reiniciar un servicio docker compose restart  
backend # Parar un servicio docker compose stop frontend
```

IMPORTANTE: NUNCA usar `docker compose down -v` en producción sin backup previo. Esto destruye TODOS los datos incluyendo la base de datos.

4.6 Volumen Docker

Volumen	Contenido	Critico	Backup
postgres_data	Base de datos completa	SI	pg_dump diario
redis_data	Cache y sesiones	No	Regenerable
media_data	Videos, HLS, thumbnails	SI	rsync/tar
tailscale_data	Estado VPN	No	Regenerable
apk_output	APK compilada	No	Regenerable
gradle_cache	Cache de Gradle	No	Regenerable

4.7 Modo Desarrollo

```
# Usar docker-compose.dev.yml (expone puertos adicionales) docker compose -f
docker-compose.yml -f docker-compose.dev.yml up -d # Esto expone: # PostgreSQL en
localhost:5432 # Redis en localhost:6379 # Backend en localhost:8080
```

5. Backend (Go/Fiber)

5.1 Estructura

```
backend/ cmd/server/main.go # Punto de entrada internal/ config/config.go # Carga de env vars database/database.go # Conexion PostgreSQL + Redis model/ # 16 modelos GORM repository/ # Queries a BD service/ # Logica de negocio handler/ # Controladores HTTP router/router.go # Definicion de rutas middleware/ # Auth, CORS, Rate Limit cache/cache.go # Servicio Redis ws/hub.go # WebSocket hub metrics/metrics.go # Prometheus counters util/ # JWT, Logger, Validacion Dockerfile # Multi-stage build go.mod # Dependencias
```

5.2 Desarrollo Local con Docker

Go NO se instala localmente. Todo se ejecuta via Docker:

```
# Compilar y verificar MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd):/app" \ -w /app/backend golang:1.22 go build ./... # Ejecutar tests MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd):/app" \ -w /app/backend golang:1.22 go test ./... # Ejecutar vet (linter) MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd):/app" \ -w /app/backend golang:1.22 go vet ./...
```

5.3 API Endpoints Completa

Autenticacion (Publicas)

Metodo	Ruta	Descripcion
POST	/api/v1/auth/login	Login (devuelve access + refresh token)
POST	/api/v1/auth/refresh	Renovar access token
POST	/api/v1/auth/logout	Cerrar sesion (invalida refresh token)
GET	/api/v1/auth/me	Datos del usuario autenticado

Contenido de Usuario (Protegidas)

Metodo	Ruta	Descripcion
GET	/api/v1/channels	Listar canales activos
GET	/api/v1/channels/:id	Detalle de canal
GET	/api/v1/vod	Listar peliculas/VOD
GET	/api/v1/vod/:id	Detalle de VOD
GET	/api/v1/series	Listar series
GET	/api/v1/series/:id	Detalle de serie
GET	/api/v1/series/:id/episodes	Episodios de una serie

Metodo	Ruta	Descripcion
GET	/api/v1/categories	Categorias por tipo
GET	/api/v1/search	Busqueda full-text global
GET	/api/v1/favorites	Listar favoritos del usuario
POST	/api/v1/favorites/toggle	Agregar/quitar favorito
GET	/api/v1/history	Historial de reproduccion
GET	/api/v1/history/continue	Continuar viendo
POST	/api/v1/history	Registrar posicion de reproduccion
DELETE	/api/v1/history/:id	Eliminar entrada del historial
GET	/api/v1/emissions/live	Canales con emision en vivo
GET	/api/v1/epg	Guia electronica de programacion
PUT	/api/v1/profile	Actualizar perfil
PUT	/api/v1/profile/password	Cambiar password

Administracion (Admin Only)

Recurso	Rutas	Operaciones
Dashboard	/admin/dashboard/stats	GET estadísticas
Canales	/admin/channels[:id]	CRUD + streams
VOD	/admin/vod[:id]	CRUD + enrich TMDB
Series	/admin/series[:id]	CRUD + enrich TMDB
Categorías	/admin/categories[:id]	CRUD + by-type
Usuarios	/admin/users[:id]	CRUD
EPG	/admin/epg[:id]	CRUD
IPTV	/admin/iptv/*	Import M3U, status, delete
Media	/admin/media/*	Upload, list, diagnostics
Library	/admin/library/*	Scan, import, TMDB search
Emisiones	/admin/channels/:id/emission/*	Start/stop/status FFmpeg
Playlist	/admin/channels/:id/playlist/*	Items, reorder, generate
Tailscale	/admin/tailscale/*	Status, start/stop/restart

Infraestructura

Metodo	Ruta	Descripcion
GET	/health	Health check (DB + Redis + uptime)
GET	/api/version	Version del backend
GET	/metrics	Metricas Prometheus
WS	/ws?token=JWT	WebSocket eventos en tiempo real
GET	/api/internal/validate-stream-token	Validacion interna (nginx)

Xtream Codes (Compatibilidad IPTV)

Metodo	Ruta	Descripcion
GET	/player_api.php	API Xtream Codes completa
GET	/xmltv.php	EPG en formato XMLTV
GET	/get.php	Endpoint de stream
GET	/live/:user/:pass/:id	Stream en vivo
GET	/movie/:user/:pass/:id	Stream VOD

Metodo	Ruta	Descripcion
GET	/series/:user/:pass/:id	Stream serie

5.4 Modelos de Datos

El backend define 16 modelos GORM con migracion automatica:

Modelo	Tabla	Descripcion
User	users	Usuarios (admin/user) + bcrypt hash
Session	sessions	Refresh tokens activos
Channel	channels	Canales de TV
Stream	streams	URLs de stream por canal
Category	categories	Categorias (channel/vod/series)
VOD	vods	Peliculas y videos on demand
Series	series	Series de TV
Episode	episodes	Episodios de series
Favorite	favorites	Favoritos por usuario
WatchHistory	watch_histories	Historial + posicion
EPGSource	epg_sources	Fuentes EPG (XMLTV)
EPGProgram	epg_programs	Programas de la guia
LocalMedia	local_media	Archivos multimedia locales
Playlist	playlists	Playlists de canales
PlaylistItem	playlist_items	Items de playlist
Emission	emissions	Emisiones FFmpeg activas

6. Frontend (Next.js)

6.1 Estructura

```
frontend/ src/ app/ (auth)/login/ # Pagina de login (user)/ home/ # Dashboard del
usuario channels/[id]/ # Reproductor de canal en vivo vod/[id]/ # Reproductor de
pelicula series/[id]/ # Detalle + episodios favoritos/ # Contenido favorito history/ #
Historial de reproduccion guide/ # Guia EPG settings/ # Configuracion de usuario admin/
channels/ # CRUD canales iptv/ # Importar M3U vod/ # CRUD peliculas series/ # CRUD
series categories/ # CRUD categorias library/ # Escaner de medios epg/ # Fuentes EPG
users/ # CRUD usuarios tailscale/ # Estado VPN components/ui/ # Componentes
reutilizables context/ # AuthContext + ToastContext lib/ # API, utils, i18n, websocket
__tests__/ # 88 suites, 1434 tests
```

6.2 Desarrollo Local

```
# Instalar dependencias cd frontend npm install # Modo desarrollo (hot reload) npm run
dev # Abre en http://localhost:3000 # Build de produccion npm run build npm start
```

6.3 Dependencias Principales

Paquete	Version	Uso
next	14.2.29	Framework React con App Router
react / react-dom	18.x	UI Library
tailwindcss	3.4.17	CSS utility-first
axios	1.8.4	Cliente HTTP
hls.js	1.6.0	Reproductor HLS (web)
i18next	24.2.2	Internacionalizacion
lucide-react	0.475.0	Iconos
clsx + tailwind-merge	-	Utilidades CSS

6.4 Características Especiales

- **PWA:** Service Worker para funcionamiento offline (manifest + sw.js)
- **i18n:** Soporte completo español/inglés via i18next
- **WebSocket:** Eventos en tiempo real (actualizaciones de canales, emisiones)
- **Responsive:** Diseño adaptativo para móvil, tablet y desktop
- **Accesibilidad:** Componentes con roles ARIA, navegación por teclado
- **Video Player:** Controles personalizados, calidad adaptativa, PiP

7. Android (Kotlin/Compose)

7.1 Estructura

```
android/app/src/main/java/com/tivify/app/ TivifyApp.kt # Application class (Hilt) data/
api/ TivifyApi.kt # Retrofit interface AuthInterceptor.kt BaseUrlInterceptor.kt
UnauthorizedInterceptor.kt di/ AppModule.kt # Hilt: Retrofit, DataStore, etc. ui/ login/
# Login screen + ViewModel home/ # Dashboard + ViewModel channels/ # Lista + detalle +
ViewModel player/ # ExoPlayer + controles vod/ # Peliculas + ViewModel series/ # Series
+ ViewModel favorites/ # Favoritos + ViewModel history/ # Historial + ViewModel epg/ #
Guia EPG + ViewModel profile/ # Perfil usuario about/ # Acerca de navigation/ # NavHost
+ rutas splash/ # Splash screen theme/ # Material 3 theme
```

7.2 Generar APK con Docker

IMPORTANTE: Seguir SIEMPRE estos pasos exactos. No usar atajos.

Paso 1: Sincronizar VERSION

```
cp VERSION android/VERSION
```

Paso 2: Build con --no-cache

```
docker build --no-cache -t tivify-android -f android/Dockerfile android/
```

SIEMPRE usar --no-cache para evitar servir APKs de capas cacheadas.

Paso 3: Extraer APK con docker cp

```
docker create --name tivify-extract tivify-android docker cp
tivify-extract:/app/app/build/outputs/apk/debug/app-debug.apk \ ./tivify-v2.4.0.apk
docker rm -f tivify-extract
```

IMPORTANTE: NUNCA usar docker run -v para extraer. Los volume mounts de Windows a Linux no funcionan correctamente y entregan APKs viejas.

Paso 4: Verificar APK

```
docker create --name verify-apk tivify-android bash -c \
'/opt/android-sdk/build-tools/35.0.0/aapt dump badging \ /tmp/app.apk 2>/dev/null | grep
-E "versionCode|versionName"; \ md5sum /tmp/app.apk; \ md5sum
/app/app/build/outputs/apk/debug/app-debug.apk' docker cp ./tivify-v2.4.0.apk
verify-apk:/tmp/app.apk docker start -a verify-apk docker rm -f verify-apk
```

Verificar que: versionName coincide con VERSION, y los checksums MD5 son identicos.

Paso 5: Build via Docker Compose (alternativa)

```
cd docker docker compose --profile build-apk up android-build # La APK queda en el
volumen apk_output
```

7.3 Configuración del Build

Parametro	Valor	Archivo
minSdk	24 (Android 7.0)	build.gradle.kts
targetSdk	35	build.gradle.kts
compileSdk	35	build.gradle.kts
Kotlin	2.0.21	build.gradle.kts
Gradle	8.11.1	gradle-wrapper.properties
AGP	8.7.3	build.gradle.kts (project)
versionCode	6 (incrementar siempre +1)	build.gradle.kts
versionName	Lee de archivo VERSION	build.gradle.kts

8. Base de Datos (PostgreSQL)

8.1 Configuración

PostgreSQL 16 Alpine con configuración optimizada:

```
# docker-compose.yml command: - "postgres" - "-c" - "max_connections=200" - "-c" -  
"shared_buffers=256MB"
```

8.2 Migraciones

GORM ejecuta migraciones automáticas al arrancar el backend. El archivo `docker/postgres/init.sql` contiene la configuración inicial de la base de datos.

8.3 Acceso a la Base de Datos

```
# Abrir psql dentro del contenedor docker exec -it tivify-postgres psql -U tivify -d  
tivify # Consultas utiles \dt -- Listar tablas \d+ users -- Estructura de tabla users  
SELECT count(*) FROM users; -- Contar usuarios SELECT count(*) FROM channels; -- Contar  
canales \q -- Salir
```

8.4 Backup

```
# Backup completo docker exec tivify-postgres pg_dump -U tivify tivify > \ backup_$(date  
+%Y%m%d_%H%M%S).sql # Backup comprimido docker exec tivify-postgres pg_dump -U tivify  
-Fc tivify > \ backup_$(date +%Y%m%d).dump # Backup automatizado (cron) 0 2 * * * docker  
exec tivify-postgres pg_dump -U tivify tivify | \ gzip > /backups/tivify_$(date  
+%Y%m%d).sql.gz
```

8.5 Restauración

```
# Desde SQL plano cat backup.sql | docker exec -i tivify-postgres psql -U tivify -d  
tivify # Desde dump binario docker exec -i tivify-postgres pg_restore -U tivify -d  
tivify < backup.dump # Restauración completa (recrear BD) docker exec tivify-postgres  
dropdb -U tivify tivify docker exec tivify-postgres createdb -U tivify tivify cat  
backup.sql | docker exec -i tivify-postgres psql -U tivify -d tivify
```

IMPORTANTE: Siempre hacer backup ANTES de restaurar. La restauración sobrescribe datos existentes.

9. Redis y Cache

9.1 Configuración

Redis 7 Alpine con autenticación por password:

```
# docker-compose.yml command: redis-server --requirepass ${REDIS_PASSWORD} # Limite de memoria: 256 MB # Volumen: redis_data:/data
```

9.2 Uso en la Aplicación

Funcion	Clave	TTL
Sesiones (refresh tokens)	session:*	168h (7 dias)
Cache de categorias	categories:*	10 min
WebSocket pub/sub	ws:*	-

9.3 Comandos Utiles

```
# Conectar a Redis docker exec -it tivify-redis redis-cli -a ${REDIS_PASSWORD} # Ver todas las claves KEYS * # Ver info de memoria INFO memory # Flush cache (no afecta sesiones) DEL categories:channel categories:vod categories:series # Flush TODO (cierra todas las sesiones) FLUSHALL # Monitor de comandos en tiempo real MONITOR
```

10. Nginx y Proxy Inverso

10.1 Arquitectura

nginx actua como punto de entrada unico, distribuyendo trafico al backend y frontend segun la ruta:

```
/ -> frontend:3000 (Next.js) /api/* -> backend:8080 (Go API) /ws -> backend:8080 (WebSocket) /health -> backend:8080 (Health check) /metrics -> backend:8080 (Prometheus) /media/* -> /media/ (Archivos estaticos) /library/* -> /library/ (Biblioteca local) /player_api.php -> backend:8080 (Xtream Codes) /tivify.apk -> /output/ (APK download)
```

10.2 Rate Limiting

Zona	Limite	Aplicado a
auth_limit	5 req/min	/api/v1/auth/*
api_limit	30 req/s (burst 50)	/api/*
upload_limit	5 req/min	Uploads de archivos

10.3 Headers de Seguridad

```
X-Content-Type-Options: nosniff X-Frame-Options: DENY X-XSS-Protection: 1; mode=block Content-Security-Policy: default-src 'self'; ... Strict-Transport-Security: max-age=31536000 Permissions-Policy: camera=(), microphone=(), geolocation=()
```

10.4 Archivos de Configuracion

```
nginx/ Dockerfile # Build de imagen nginx nginx.conf # Configuracion principal conf.d/ default.conf # Rutas, upstream, seguridad (282 lineas)
```

10.5 Recargar Configuracion

```
# Sin reiniciar (reload graceful) docker exec tivify-nginx nginx -s reload # Verificar config antes de recargar docker exec tivify-nginx nginx -t
```

11. Tailscale VPN

11.1 Funcion

Tailscale proporciona acceso remoto seguro al servidor sin necesidad de abrir puertos ni configurar port forwarding. Crea una red VPN encriptada (WireGuard) entre tus dispositivos.

11.2 Configuracion

```
# En .env TS_AUTHKEY=tskey-auth-xxxxx # Obtener en login.tailscale.com
TS_HOSTNAME=tivify # Nombre en la tailnet TS_SERVE_MODE=https # https = certificados
automaticos ENABLE_TAILSCALE=true
```

11.3 Gestion desde el Panel Admin

El panel de administracion incluye controles para Tailscale:

Accion	Endpoint	Descripcion
Ver estado	GET /admin/tailscale/status	IP, hostname, conectado
Iniciar	POST /admin/tailscale/start	Arrancar contenedor
Parar	POST /admin/tailscale/stop	Detener contenedor
Reiniciar	POST /admin/tailscale/restart	Restart contenedor

11.4 Acceso Remoto

```
# Acceder desde cualquier dispositivo en tu tailnet: https://tivify.tu-tailnet.ts.net #
La app Android debe configurar la URL del servidor a: https://tivify.tu-tailnet.ts.net
```

11.5 Arquitectura de Red

```
Tailscale Container | network_mode: "service:nginx" | Comparte la red con nginx ->
Tráfico entra por Tailscale -> nginx lo distribuye como si fuera tráfico local
```

12. Testing y Coverage

12.1 Frontend (Jest + React Testing Library)

```
# Ejecutar todos los tests cd frontend npx jest --no-coverage # Con coverage npx jest --coverage # Tests de un archivo especifico npx jest __tests__/lib/api.test.ts # Tests por patron npx jest --testPathPattern="admin" # Watch mode (re-ejecuta al cambiar) npx jest --watch
```

Estadísticas Actuales

Metrica	Valor
Test suites	88
Tests totales	1,434
Lineas cubiertas	99.08%
Statements	97.74%
Branches	88.15%
Functions	97.70%

12.2 Backend (Go Test via Docker)

```
# Ejecutar todos los tests del backend MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd):/app" \ -w /app/backend golang:1.22 go test ./... # Con verbose MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd):/app" \ -w /app/backend golang:1.22 go test -v ./... # Coverage MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd):/app" \ -w /app/backend golang:1.22 go test -cover ./... # Un paquete especifico MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd):/app" \ -w /app/backend golang:1.22 go test ./internal/service/...
```

12.3 Android (Gradle)

```
# Tests unitarios ./gradlew testDebugUnitTest # Via Docker docker run --rm -v "$(pwd)/android:/app" -w /app \ gradle:8.11-jdk17 ./gradlew testDebugUnitTest
```

12.4 Linting y Analisis Estatico

```
# Go vet MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd):/app" \ -w /app/backend golang:1.22 go vet ./... # Frontend lint cd frontend && npm run lint # TypeScript check cd frontend && npx tsc --noEmit
```

13. Gestion de Versiones

13.1 Archivo VERSION

El archivo VERSION en la raiz del proyecto contiene la version semver (MAJOR.MINOR.PATCH). Actualmente: **2.4.0**

13.2 Proceso de Incremento

```
# 1. Actualizar VERSION en la raiz echo "2.5.0" > VERSION # 2. Incrementar versionCode en android/app/build.gradle.kts # Buscar: versionCode = 6 -> versionCode = 7 # (siempre +1, nunca decrementar) # 3. Copiar VERSION a android/ antes de build cp VERSION android/VERSION
```

13.3 Donde se Usa la Version

Componente	Fuente	Como se lee
Backend	internal/version/	Compilada con ldflags
Frontend	package.json	npm version
Android	VERSION + build.gradle.kts	Lee archivo VERSION en build
Docker	VERSION	ARG en Dockerfile
API	GET /api/version	Endpoint publico
Health	GET /health	Campo version en JSON

13.4 Versionado Semantico

- **MAJOR:** Cambios incompatibles en la API
- **MINOR:** Nuevas funcionalidades retrocompatibles
- **PATCH:** Correcciones de bugs

14. Operaciones Diarias

14.1 Verificacion de Salud

```
# Health check completo curl -s http://localhost/health | python3 -m json.tool #  
Respuesta esperada: # { # "status": "ok", # "database": "ok", # "redis": "ok", #  
"uptime": "72h30m15s", # "version": "2.4.0" # } # Estado de contenedores cd  
/opt/tivify/docker docker compose ps
```

14.2 Monitoreo de Logs

```
# Logs en tiempo real (todos) docker compose logs -f # Solo errores del backend docker  
compose logs -f backend 2>&1 | grep -i error # Ultimas 50 lineas de un servicio docker  
compose logs --tail=50 nginx
```

14.3 Metricas Prometheus

```
# Acceder a metricas curl -s http://localhost/metrics # Metricas disponibles: #  
http_requests_total{method, path, status} # http_request_duration_seconds{method, path}  
# active_connections # ...counters custom por handler
```

14.4 Mantenimiento

```
# Limpiar imagenes Docker no usadas docker system prune -f # Limpiar cache de Redis  
docker exec tivify-redis redis-cli -a $REDIS_PASSWORD FLUSHDB # Verificar espacio en  
disco df -h docker system df
```

15. Backups y Restauracion

15.1 Que Datos Son Criticos

Dato	Ubicacion	Prioridad	Metodo
Base de datos	postgres_data volume	CRITICA	pg_dump
Archivos media	media_data volume	CRITICA	rsync/tar
Configuracion	.env	ALTA	Copia manual
Codigo fuente	Repositorio Git	ALTA	git push
Cache Redis	redis_data volume	BAJA	Regenerable

15.2 Script de Backup Completo

```
#!/bin/bash BACKUP_DIR="/backups/tivify/$(date +%Y%m%d)" mkdir -p $BACKUP_DIR # 1. Base de datos docker exec tivify-postgres pg_dump -U tivify -Fc tivify > \ $BACKUP_DIR/database.dump # 2. Media files docker run --rm -v tivify_media_data:/data:ro \ -v $BACKUP_DIR:/backup alpine \ tar czf /backup/media.tar.gz -C /data . # 3. Configuracion cp /opt/tivify/.env $BACKUP_DIR/env.backup # 4. Limpiar backups antiguos (30 dias) find /backups/tivify -maxdepth 1 -mtime +30 -exec rm -rf {} \; echo "Backup completado en $BACKUP_DIR"
```

15.3 Restauracion Completa

```
# 1. Parar servicios cd /opt/tivify/docker docker compose down # 2. Restaurar base de datos docker compose up -d postgres sleep 10 # Esperar que arranque docker exec -i tivify-postgres pg_restore -U tivify \ -d tivify --clean < /backups/tivify/20260318/database.dump # 3. Restaurar media docker run --rm -v tivify_media_data:/data \ -v /backups/tivify/20260318:/backup alpine \ sh -c "rm -rf /data/* && tar xzf /backup/media.tar.gz -C /data" # 4. Restaurar .env cp /backups/tivify/20260318/env.backup /opt/tivify/.env # 5. Levantar todo docker compose up -d
```


16. Streaming y Media

16.1 Flujo de Streaming

```
Fuente (URL/archivo) -> FFmpeg -> Segmentos HLS (.ts + .m3u8) | /media/hls/ | nginx  
(servir estatico) | hls.js (web) / ExoPlayer (Android)
```

16.2 Transcodificacion FFmpeg

Configuracion via variables de entorno:

Variable	Opciones	Default
FFMPEG_PRESET	ultrafast, superfast, veryfast, faster, fast, medium, slow	faster
FFMPEG_HWACCEL	nvidia (NVENC), intel (QSV), none	none
FFMPEG_AUDIO_BITRATE	128k, 192k, 256k	192k
HLS_SEGMENT_DURATION	Segundos por segmento	10
HLS_PLAYLIST_SIZE	Segmentos en playlist	5

16.3 Emisiones en Vivo

Las emisiones usan FFmpeg para convertir fuentes (URLs, archivos, playlists) en streams HLS en tiempo real:

```
# Desde el panel admin: # POST /api/v1/admin/channels/:id/emission/start # POST  
/api/v1/admin/channels/:id/emission/stop # GET  
/api/v1/admin/channels/:id/emission/status # El proceso FFmpeg corre dentro del  
contenedor backend # Los segmentos HLS se escriben en /media/hls/{channel_id}/
```

16.4 Biblioteca Local

El Library Scanner permite importar medios desde un disco externo o directorio montado:

```
# Configurar ruta en .env LIBRARY_PATH=/mnt/usb # Desde el panel admin: # POST  
/api/v1/admin/library/scan # GET /api/v1/admin/library/scan:sessionId/status # POST  
/api/v1/admin/library/import # POST /api/v1/admin/library/tmdb/search (enriquecer  
metadata)
```

16.5 Upload de Media

```
# Upload directo de archivo POST /api/v1/admin/media/upload Content-Type:  
multipart/form-data Body: file=@pelicula.mp4 # Upload + crear VOD automaticamente POST  
/api/v1/admin/media/upload-vod Content-Type: multipart/form-data Body:  
file=@pelicula.mp4, title=Mi Pelicula # Rate limit: 5 uploads por minuto por usuario
```

16.6 Compatibilidad Xstream Codes

TIVIFY es compatible con reproductores IPTV que usan el protocolo Xstream Codes. Esto permite usar apps como IPTV Smarters, TiviMate, etc.

```
# Datos de conexion para reproductores IPTV: URL: http://tu-servidor Username: (usuario TIVIFY) Password: (password TIVIFY) # Endpoints compatibles: /player_api.php - API principal /xmtv.php - Guia EPG /live/user/pass/ - Streams en vivo /movie/user/pass/ - VOD /series/user/pass/- Series
```

17. Troubleshooting

17.1 Contenedor No Arranca

```
# Ver logs del contenedor que falla docker compose logs backend # Ver estado detallado
docker inspect tivify-backend | grep -A 10 "State" # Causa comun: PostgreSQL no esta
listo # Solucion: Verificar health checks docker compose ps
```

17.2 Error de Conexion a BD

```
# Verificar que PostgreSQL esta corriendo docker exec tivify-postgres pg_isready -U
tivify # Verificar credenciales en .env grep DB_ .env # Ver logs de PostgreSQL docker
compose logs postgres
```

17.3 Error de Conexion a Redis

```
# Verificar Redis docker exec tivify-redis redis-cli -a $REDIS_PASSWORD ping # Respuesta
esperada: PONG # Ver memoria usada docker exec tivify-redis redis-cli -a $REDIS_PASSWORD
INFO memory
```

17.4 Frontend No Carga

```
# Verificar que el frontend esta corriendo docker compose logs frontend # Verificar
variables de entorno docker exec tivify-frontend printenv | grep NEXT_PUBLIC # Problema
comun: BASE_URL incorrecto # NEXT_PUBLIC_API_URL debe apuntar a la URL publica
```

17.5 Streaming No Funciona

```
# Verificar FFmpeg docker exec tivify-backend ffmpeg -version # Verificar permisos del
volumen media docker exec tivify-backend ls -la /media/ # Verificar que nginx sirve los
segmentos HLS curl -I http://localhost/media/hls/test/index.m3u8 # Ver procesos FFmpeg
activos docker exec tivify-backend ps aux | grep ffmpeg
```

17.6 APK Build Falla

```
# Verificar que VERSION existe cat VERSION cat android/VERSION # Debe ser igual # Build
con output verbose docker build --no-cache --progress=plain \ -t tivify-android -f
android/Dockerfile android/ 2>&1 | tee build.log # Causa comun: memoria insuficiente
para Gradle # Solucion: Asignar al menos 4GB de RAM a Docker
```

17.7 Tailscale No Conecta

```
# Ver logs de Tailscale docker compose logs tailscale # Verificar auth key # Las claves
efimeras expiran en 24h # Generar nueva en:
https://login.tailscale.com/admin/settings/keys # Verificar que /dev/net/tun existe ls
-la /dev/net/tun
```

18. Referencia Rapida

18.1 Comandos Esenciales

Accion	Comando
Levantar todo	cd docker && docker compose up -d
Parar todo	cd docker && docker compose down
Ver estado	cd docker && docker compose ps
Ver logs	docker compose logs -f [servicio]
Rebuild backend	docker compose up -d --build backend
Rebuild frontend	docker compose up -d --build frontend
Rebuild todo	docker compose up -d --build --force-recreate
Health check	curl http://localhost/health
Backup BD	docker exec tivify-postgres pg_dump -U tivify tivify > backup.sql
Acceso psql	docker exec -it tivify-postgres psql -U tivify -d tivify
Acceso Redis	docker exec -it tivify-redis redis-cli -a \$REDIS_PASSWORD
Frontend tests	cd frontend && npx jest
Backend tests	MSYS_NO_PATHCONV=1 docker run --rm -v "\$(pwd):/app" -w /app/backend golang:1.22 go test ./...
Build APK	cp VERSION android/VERSION && docker build --no-cache -t tivify-android -f android/Dockerfile android/
Limpiar Docker	docker system prune -f

18.2 URLs Importantes

URL	Descripcion
http://servidor/	Frontend (interfaz web)
http://servidor/admin	Panel de administracion
http://servidor/login	Pagina de login
http://servidor/health	Health check (JSON)
http://servidor/metrics	Metricas Prometheus
http://servidor/api/version	Version del backend
http://servidor/tivify.apk	Descarga de APK Android

URL	Descripcion
ws://servidor/ws?token=JWT	WebSocket tiempo real

18.3 Credenciales por Defecto

IMPORTANTE: CAMBIAR todas las credenciales por defecto antes de desplegar en produccion.

Servicio	Usuario	Password	Fuente
Admin TIVIFY	admin	admin123	.env (ADMIN_*)
PostgreSQL	tivify	changeme	.env (DB_*)
Redis	-	changeme	.env (REDIS_PASSWORD)

18.4 Limites de Recursos

Servicio	CPU	RAM	Notas
PostgreSQL	1 core	1024 MB	max_connections=200, shared_buffers=256MB
Redis	0.5 core	256 MB	Persistencia en disco
Backend	1 core	512 MB	Incluye FFmpeg
Frontend	0.5 core	256 MB	Next.js SSR
nginx	1 core	256 MB	Rate limiting, static files
TOTAL	4 cores	~2.3 GB	Minimo recomendado